

AIDA positioning

AIDA positioning · best-effort comparison — live source:
github.com/joemooney/aida

rendered 2026-07-10

“Should I use AIDA or X?” comes up every time a new developer maps the AI/dev-tools ecosystem. The honest answer is usually “**different scopes; sometimes both.**” This directory holds the sharp, paired comparisons that make that answer concrete.

Sister documents:

- [README.md](#) — the elevator pitch: AIDA’s defensible niche in eight bullets.
- [OVERVIEW.md](#) — the big-picture vision.
- [WHY-AIDA.md](#) — narrative for “why does this exist?”
- [competitive-analysis/](#) — the living landscape scan + ecosystem tracking (dated snapshots and per-topic files).

The docs here are **focused comparisons**: one file per neighbor, each answering “when AIDA, when X, when both?” in one sitting.

The defensible niche statement

AIDA is **the agent-collaboration layer for project intent**. Not a replacement for project management tools, not a replacement for code review tools, not a replacement for documentation generators. The defensible thing AIDA brings is the *stable, queryable graph of what exists, who depends on it, and why* — served to AI through MCP and to humans through a small CLI.

Karpathy-style “structured markdown queryable by Claude” is the floor; Spec Kit’s spec-driven workflow is the ceiling. AIDA sits between them as the **durable index** — a small invisible kernel that captures what exists, plus optional layered modules for everything else.

— [OVERVIEW.md](#)

Everything in this directory is in service of making that scope honest. If a comparison reads as “*AIDA replaces X*,” it’s probably wrong. The right framing is almost always “*AIDA composes with X, and here’s the seam.*”

Index

File	The question it answers
vs-spec-kit.md	(nearest competitor) When GitHub Spec Kit's first-feature spec→plan→tasks scaffold is enough vs when you need a maintained, cross-cutting graph: stable IDs, typed relationships, trace enforcement, lifecycle, MCP — over the project's whole life.
vs-kiro.md	(nearest competitor) When AWS Kiro's polished agentic IDE with EARS-notation requirements + per-feature task→requirement traceability is enough vs when you need a vendor-neutral, git-canonical graph readable by any agent via MCP — independent of the editor that produced the specs.
vs-ultrareview.md	When to reach for AIDA's /aida-review (free, integrated, lifecycle-aware) vs Claude Code's /ultrareview (paid/quota, multi-agent depth).
vs-ultraplan.md	When /ultraplan's dense LLM-generated planning brief earns its keep vs AIDA's persistent, graph-integrated plan files. Sister doc to vs-ultrareview.
vs-claude-code-subagents.md	Why AIDA's roles aren't a reinvention of Claude Code's /agents — within-conversation primitive vs cross-conversation workflow layer, and how they compose.
vs-claude-code-workflows.md	When Claude Code's /workflows (within-task JS orchestration — fan out subagents, hold the plan in code, end with one answer) fits vs AIDA's cross-session graph + drain (a spec's lifecycle across sessions/vendors). Different units of work; they compose. Companion to vs-claude-code-subagents.md.
vs-agent-teams.md	When Claude Code's Agent Teams (within-session multi-agent coordination — shared mailbox, self-claim task-list, file-locking, auto-unblocking dependencies, plan-approval gate) fits vs AIDA's cross-session, cross-vendor graph + lifecycle. The closest provider overlap yet — on the <i>coordination</i> layer — and why the gap persists on incentive, not capability. Companion to vs-claude-code-subagents.md + vs-claude-code-workflows.md.
vs-karpathy.md.md	When structured markdown alone is enough vs when you actually need a relationship graph + stable IDs + MCP server.
vs-saas-pm.md	When Linear / Jira / GitHub Projects make sense vs the lightweight git-canonical, code-aware angle AIDA serves.

File	The question it answers
vs-aider.md	(adjacent neighbor, not competitor) Aider is a terminal pair-programmer that auto-commits every change; AIDA is the spec graph + lifecycle above the editing. Different layers — “Aider edits, AIDA remembers why” — and how to run Aider as the implementer inside AIDA.
vs-continue.md	(adjacent neighbor, not competitor) Continue is a CI-native AI assistant with declarative <code>.continue/checks/</code> markdown gates; AIDA is the requirement graph + lifecycle. Continue enforces <i>how</i> code looks; AIDA remembers <i>what</i> it was for. How they layer.
vs-langgraph.md	(runtime/control-plane neighbor) LangGraph is a durable execution runtime for stateful agent workflows; AIDA is the program-owned coordination record those workflows can read from and write back to. “LangGraph runs the agents; AIDA remembers what the work is for.”
vs-a2a.md	(standards/transport neighbor, not competitor) A2A/MCP/ACP are stateless agent $\leftrightarrow$ agent (and agent $\leftrightarrow$ tool) <i>transports</i> ; AIDA is the durable, multi-vendor-readable coordination <i>record</i> they deliberately leave open. “A2A carries the live handoff; AIDA holds the record that survives it.” Both, different layers.
vs-axi.md	(autonomy/ergonomics-layer neighbor) Kun Chen’s AXI family (tasks-axi / firstmate / gn timer) — single-objective, agent-native overnight loops with token-efficient output; AIDA is the spec-graph-driven drain with stable IDs, traces, lifecycle, and cross-vendor coordination. AXI’s output benchmark was validation AIDA acted on (TOON/AIDA_AGENT_OUTPUT, SPIKE-73). Wins/loses stated per axis.

Cross-cutting decision aids

Not one-neighbor-at-a-time comparisons, but the questions that span all of them:

File	The question it answers
when-not-to-use-aida.md	The honest scope limits — six cases where a neighbor tool alone is the right call, and AIDA’s overhead wouldn’t earn its keep. Read this <i>first</i> if you’re deciding whether to adopt at all.

File	The question it answers
agent-decision-matrix.md	The build-vs-buy-vs-wait aid for “ <i>which agent runtime, and how much workflow do I push into vendor-neutral infrastructure?</i> ” — Claude Code vs Codex CLI vs an AIDA-style substrate, axis by axis. Includes the honest “do not adopt a substrate when...” rows. Grounded in the Claude→Codex migration docs under docs/agents/.
composition.md	The recipe book for “ <i>use AIDA with X</i> ” — Spec Kit, Agent Teams, MCP editors, /workflow, GitHub Issues, Karpathy markdown. Names the seam (and where a bridge is still manual today) for each.

Future-work files mentioned in [STORY-107](#) and not yet seeded:

- `vs-github-projects.md` (separate from `vs-saas-pm.md` if/when the GitHub-specific angle warrants its own page)
- `vs-cursor.md` (AI code-editor neighbor — composition story; deferred until there’s a verified Cursor scan to ground it, since Cursor’s proprietary surface moves fast)
- `vs-mdbook.md`, `vs-docusaurus.md` (documentation tooling — `aida docs build projects` to one of these, not a substitute)

Maintenance rhythm

Positioning rots fast. Each of these docs has a Last updated date in its frontmatter — when that date is older than the comparison target’s most recent meaningful release, the doc is suspect.

When to update

- **Competing/composing tool ships a major feature.** Example: Claude Code adds /ultrareview → re-evaluate `vs-ultrareview.md`. Tool X adds an MCP server → re-evaluate the relevant `vs-X.md`.
- **AIDA ships a release that changes the comparison.** Example: AIDA’s MCP server gains a tool that Tool X already had → the gap narrows, update the page. Composes with EPIC-25’s release-subtask model: every AIDA release that meaningfully shifts positioning should file a release subtask “review docs/positioning/ for changes” against the candidate tools.
- **A user asks the comparison question and the doc didn’t answer it well.** The conversation that prompted the gap is the freshest possible doc seed — capture it via `aida doc add --about <relevant-spec> --scenario "positioning"`.

Who triggers it

- **Anyone** who notices a gap. The friction is *open the file and edit* — no PR template required, no review gate beyond “is it accurate?”

- **Release manager** on each AIDA release: sweep docs/positioning/ for staleness as one of the release subtasks.
- **Agent prompt**, when a user mentions a neighbor tool in conversation: *“AIDA has a positioning doc for X — docs/positioning/vs-X.md. Reading it before answering will keep the response calibrated. Capture anything new the user surfaces as a doc seed.”*

How to update

1. Edit the .md file directly. Bump the Last updated line.
2. If the update is substantive (new capability invalidates an old claim), file an aida doc add entry referencing this page so the change is searchable from the graph.
3. Commit with a docs(positioning): scope so the change is easy to find in git log --grep.

Honest scope of this maintenance system

This rhythm assumes positioning is **best-effort**, not auditable. If you need legally accurate competitor comparisons (procurement, regulated industries) these docs are not that. They’re calibration documents — what AIDA’s authors believe is true at the date stamped. Pricing, feature parity, and roadmap claims about competing tools should be re-verified against the vendor’s own docs before any high-stakes decision.

See also

- **EPIC-24** — Living documentation: positioning docs are themselves Doc entries in the graph; this directory is the prose rendering.
- **STORY-107** — The story this directory implements.
- **STORY-104** — aida doc data model that lets future positioning seeds land via the graph first.
- **STORY-105** — /aida-doc proactive capture skill — the producer side of those seeds.